

Machine Learning Lab

Assessment - 2

Name : Arjun Krishna B .

Reg No : 23BCA0050

1. Implement a Python program to handle missing values in a dataset

Program :

```
import pandas as pd
import numpy as np

# Sample dataset with missing values
data = {
    'Name': ['Arjun', 'Krishna', 'Raj', 'Gopal', np.nan],
    'Age': [19, np.nan, 12, 21, 90],
    'City': ['Kerala', np.nan, 'Chennai', 'Mumbai', 'Agra']
}

# Create a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame with Missing Values:")
print(df)

# 1. Check for missing values in the dataset
print("\nMissing values in the dataset:")
print(df.isnull().sum())

# 2. Handle missing values:
# a) Remove rows with any missing values
df_dropped = df.dropna()
print("\nDataFrame after dropping rows with missing values:")
```

```

print(df_dropped)

# b) Fill missing values with a specific value (e.g., 'Unknown' for strings and mean for numerical
columns)

df_filled = df.copy() # Create a copy to avoid modifying the original
df_filled['Name'].fillna('Unknown', inplace=True)
df_filled['Age'].fillna(df_filled['Age'].mean(), inplace=True) # Fill with the mean of the column
df_filled['City'].fillna('Unknown', inplace=True)

print("\nDataFrame after filling missing values:")
print(df_filled)

# c) Replace missing values with a value based on a condition or some other method
df_replaced = df.copy()
df_replaced['Age'].replace(np.nan, df['Age'].mean(), inplace=True) # Replace NaN with mean for Age
column

print("\nDataFrame after replacing missing values:")
print(df_replaced, "arjun krishna \n\n 23BCA0050");

```

OUTPUT :

1. Write a Python script to encode categorical variables using techniques like One-Hot Encoding and Label Encoding

PROGRAM :

```

import pandas as pd

from sklearn.preprocessing import LabelEncoder

# Sample DataFrame with new categorical variables
data = {
    'Grade': ['A', 'B', 'C', 'A', 'B'],

```

```
    'Major': ['Computer Science', 'Mathematics', 'Engineering', 'Computer
Science', 'Engineering'],
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Male']
}
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# 1. One-Hot Encoding using pandas get_dummies
```

```
df_one_hot = pd.get_dummies(df, columns=['Grade', 'Major', 'Gender'])
```

```
print("\nDataFrame after One-Hot Encoding:")
```

```
print(df_one_hot)
```

```
# 2. Label Encoding using LabelEncoder from sklearn
```

```
label_encoder = LabelEncoder()
```

```
# Applying label encoding to 'Grade', 'Major', and 'Gender' columns
```

```
df['Grade_Encoded'] = label_encoder.fit_transform(df['Grade'])
```

```
df['Major_Encoded'] = label_encoder.fit_transform(df['Major'])
```

```
df['Gender_Encoded'] = label_encoder.fit_transform(df['Gender'])
```

```
print("\nDataFrame after Label Encoding:")
```

```
print(df, "\n\n23BCA0050 .")
```

OUTPUT :

```
Original DataFrame:
  Grade      Major Gender
0     A  Computer Science  Male
1     B    Mathematics  Female
2     C    Engineering  Female
3     A  Computer Science  Male
4     B    Engineering  Male

DataFrame after One-Hot Encoding:
  Grade_A Grade_B Grade_C Major_Computer Science Major_Engineering \
0     True  False  False          True          False
1    False   True  False          False          False
2    False  False   True          False          True
3     True  False  False          True          False
4    False   True  False          False          True

  Major_Mathematics Gender_Female Gender_Male
0             False          False          True
1             True          True          False
2             False          True          False
3             False          False          True
4             False          False          True

DataFrame after Label Encoding:
  Grade      Major Gender  Grade_Encoded  Major_Encoded \
0     A  Computer Science      Male          0          0
1     B    Mathematics  Female          1          2
2     C    Engineering  Female          2          1
3     A  Computer Science      Male          0          0
4     B    Engineering  Male          1          1

  Gender_Encoded
0              1
1              0
2              0
3              1
4              1

23BCA0050 .
```

2. Implement a Python program to scale the features of a dataset using StandardScaler, MinMaxScaler, and RobustScaler .

PROGRAM :

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler,
RobustScaler
```

```
# Sample DataFrame with numerical features
```

```
data = {  
    'Feature1': [14, 22, 31, 41, 5],  
    'Feature2': [7, 5, 25, 30, 32],  
    'Feature3': [10, 200, 30, 0, 500]  
}
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")  
print(df, "\n\n 23BCA0050 .")
```

```
# 1. StandardScaler: Standardizes features by removing the mean and scaling  
to unit variance
```

```
scaler_standard = StandardScaler()  
df_standard_scaled = scaler_standard.fit_transform(df)
```

```
# Convert scaled features back to DataFrame for better readability
```

```
df_standard_scaled = pd.DataFrame(df_standard_scaled,  
columns=df.columns)
```

```
print("\nDataFrame after Standard Scaling:")  
print(df_standard_scaled, "\n\n 23BCA0050 .")
```

```
# 2. MinMaxScaler: Scales features to a specified range, by default [0, 1]
```

```
scaler_minmax = MinMaxScaler()
df_minmax_scaled = scaler_minmax.fit_transform(df)

# Convert scaled features back to DataFrame
df_minmax_scaled = pd.DataFrame(df_minmax_scaled,
columns=df.columns)

print("\nDataFrame after MinMax Scaling:")
print(df_minmax_scaled, "\n\n 23BCA0050 .")
```

3. RobustScaler: Scales features using the median and interquartile range (IQR)

```
scaler_robust = RobustScaler()
df_robust_scaled = scaler_robust.fit_transform(df)

# Convert scaled features back to DataFrame
df_robust_scaled = pd.DataFrame(df_robust_scaled, columns=df.columns)

print("\nDataFrame after Robust Scaling:")
print(df_robust_scaled, "\n\n 23BCA0050 .")
```

OUTPUT :

```

➡ Original DataFrame:
   Feature1  Feature2  Feature3
0         14         7         10
1         22         5        200
2         31        25         30
3         41        30          0
4          5        32        500

23BCA0050 .

DataFrame after Standard Scaling:
   Feature1  Feature2  Feature3
0 -0.682798 -1.111741 -0.724352
1 -0.047637 -1.285451  0.272944
2  0.666919  0.451645 -0.619374
3  1.460870  0.885919 -0.776841
4 -1.397354  1.059628  1.847623

23BCA0050 .

DataFrame after MinMax Scaling:
   Feature1  Feature2  Feature3
0  0.250000  0.074074    0.02
1  0.472222  0.000000    0.40
2  0.722222  0.740741    0.06
3  1.000000  0.925926    0.00
4  0.000000  1.000000    1.00

23BCA0050 .

DataFrame after Robust Scaling:
   Feature1  Feature2  Feature3
0 -0.470588 -0.782609 -0.105263
1  0.000000 -0.869565  0.894737
2  0.529412  0.000000  0.000000
3  1.117647  0.217391 -0.157895
4 -1.000000  0.304348  2.473684

23BCA0050 .

```

3. Write a python program using sciki learn library with iris flower data set

PROGRAM :

```
import pandas as pd
```

```
from sklearn.datasets import load_iris
```

```
# Step I: Load the Iris data from sklearn and convert it into a DataFrame
```

```
iris_data = load_iris()

iris_df = pd.DataFrame(data=iris_data['data'],
                        columns=iris_data['feature_names'])

iris_df['Species'] = iris_data['target_names'][iris_data['target']]
```

Print the shape, type of the data, and the first 3 rows

```
print("Shape of the dataset:", iris_df.shape)

print("Type of the dataset:", type(iris_df))

print("\nFirst 3 rows of the dataset:")

print(iris_df.head(3), "\n\n 23BCA0050")
```

Step II: Print keys, number of rows-columns, feature names, and description

```
print("\nKeys of the dataset:", iris_data.keys())

print("Number of rows and columns:", iris_df.shape)

print("Feature names:", iris_data['feature_names'])

print("\nDescription of the Iris dataset:")

print(iris_data['DESCR'], "\n\n 23BCA0050")
```

Step III: Basic statistical details like percentile, mean, std, etc.

```
print("\nStatistical details of the Iris dataset:")

print(iris_df.describe(), "\n\n 23BCA0050")
```

Step IV: Get observations for each species (setosa, versicolor, virginica)

```
print("\nObservations for Iris Setosa:")

print(iris_df[iris_df['Species'] == 'setosa'], "\n\n 23BCA0050")
```



```

print("\nObservations for Iris Versicolor:")
print(iris_df[iris_df['Species'] == 'versicolor'], "\n\n 23BCA0050")

print("\nObservations for Iris Virginica:")
print(iris_df[iris_df['Species'] == 'virginica'], "\n\n 23BCA0050")

```

OUTPUT :

```

➞ Shape of the dataset: (150, 5)
Type of the dataset: <class 'pandas.core.frame.DataFrame'>

First 3 rows of the dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2

Species
0  setosa
1  setosa
2  setosa

23BCA0050

Keys of the dataset: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
Number of rows and columns: (150, 5)
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

```



Description of the Iris dataset:

```
.. _iris_dataset:
```

Iris plants dataset

****Data Set Characteristics:****

:Number of Instances: 150 (50 in each of three classes)

:Number of Attributes: 4 numeric, predictive attributes and the class

:Attribute Information:

- sepal length in cm
- sepal width in cm
- petal length in cm
- petal width in cm
- class:
 - Iris-Setosa
 - Iris-Versicolour
 - Iris-Virginica

:Summary Statistics:

	Min	Max	Mean	SD	Class Correlation
sepal length:	4.3	7.9	5.84	0.83	0.7826
sepal width:	2.0	4.4	3.05	0.43	-0.4194
petal length:	1.0	6.9	3.76	1.76	0.9490 (high!)
petal width:	0.1	2.5	1.20	0.76	0.9565 (high!)



:Missing Attribute Values: None
:Class Distribution: 33.3% for each of 3 classes.
:Creator: R.A. Fisher
:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
:Date: July, 1988

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken from Fisher's paper. Note that it's the same as in R, but not as in the UCI Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the pattern recognition literature. Fisher's paper is a classic in the field and is referenced frequently to this day. (See Duda & Hart, for example.) The data set contains 3 classes of 50 instances each, where each class refers to a type of iris plant. One class is linearly separable from the other 2; the latter are NOT linearly separable from each other.

.. dropdown:: References

- Fisher, R.A. "The use of multiple measurements in taxonomic problems" Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to Mathematical Statistics" (John Wiley, NY, 1950).
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis. (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.
- Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System Structure and Classification Rule for Recognition in Partially Exposed Environments". IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-2, No. 1, 67-71.
- Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions on Information Theory, May 1972, 431-433.
- See also: 1988 MLC Proceedings, 54-64. Cheeseman et al's AUTOCLASS II conceptual clustering system finds 3 classes in the data.
- Many, many more ...

Statistical details of the Iris dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
count	150.000000	150.000000	150.000000	
mean	5.843333	3.057333	3.758000	
std	0.828066	0.435866	1.765298	
min	4.300000	2.000000	1.000000	
25%	5.100000	2.800000	1.600000	
50%	5.800000	3.000000	4.350000	
75%	6.400000	3.300000	5.100000	
max	7.900000	4.400000	6.900000	

	petal width (cm)
count	150.000000
mean	1.199333
std	0.762238
min	0.100000
25%	0.300000
50%	1.300000
75%	1.800000
max	2.500000

23BCA0050



Observations for Iris Setosa:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	
5	5.4	3.9	1.7	0.4	
6	4.6	3.4	1.4	0.3	
7	5.0	3.4	1.5	0.2	
8	4.4	2.9	1.4	0.2	
9	4.9	3.1	1.5	0.1	
10	5.4	3.7	1.5	0.2	
11	4.8	3.4	1.6	0.2	
12	4.8	3.0	1.4	0.1	
13	4.3	3.0	1.1	0.1	
14	5.8	4.0	1.2	0.2	
15	5.7	4.4	1.5	0.4	
16	5.4	3.9	1.3	0.4	
17	5.1	3.5	1.4	0.3	
18	5.7	3.8	1.7	0.3	
19	5.1	3.8	1.5	0.3	
20	5.4	3.4	1.7	0.2	
21	5.1	3.7	1.5	0.4	
22	4.6	3.6	1.0	0.2	
23	5.1	3.3	1.7	0.5	
24	4.8	3.4	1.9	0.2	
25	5.0	3.0	1.6	0.2	
26	5.0	3.4	1.6	0.4	
27	5.2	3.5	1.5	0.2	
28	5.2	3.4	1.4	0.2	
29	4.7	3.2	1.6	0.2	
30	4.8	3.1	1.6	0.2	
31	5.4	3.4	1.5	0.4	
32	5.2	4.1	1.5	0.1	



33	5.5	4.2	1.4	0.2
34	4.9	3.1	1.5	0.2
35	5.0	3.2	1.2	0.2
36	5.5	3.5	1.3	0.2
37	4.9	3.6	1.4	0.1
38	4.4	3.0	1.3	0.2
39	5.1	3.4	1.5	0.2
40	5.0	3.5	1.3	0.3
41	4.5	2.3	1.3	0.3
42	4.4	3.2	1.3	0.2
43	5.0	3.5	1.6	0.6
44	5.1	3.8	1.9	0.4
45	4.8	3.0	1.4	0.3
46	5.1	3.8	1.6	0.2
47	4.6	3.2	1.4	0.2
48	5.3	3.7	1.5	0.2
49	5.0	3.3	1.4	0.2

Species

0	setosa
1	setosa
2	setosa
3	setosa
4	setosa
5	setosa
6	setosa
7	setosa
8	setosa
9	setosa
10	setosa
11	setosa
12	setosa
13	setosa
14	setosa
15	setosa



```
16 setosa
17 setosa
18 setosa
19 setosa
20 setosa
21 setosa
22 setosa
23 setosa
24 setosa
25 setosa
26 setosa
27 setosa
28 setosa
29 setosa
30 setosa
31 setosa
32 setosa
33 setosa
34 setosa
35 setosa
36 setosa
37 setosa
38 setosa
39 setosa
40 setosa
41 setosa
42 setosa
43 setosa
44 setosa
45 setosa
46 setosa
47 setosa
48 setosa
49 setosa
```

↕	Observations for Iris Versicolor:					
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\	
50	7.0	3.2	4.7	1.4		
51	6.4	3.2	4.5	1.5		
52	6.9	3.1	4.9	1.5		
53	5.5	2.3	4.0	1.3		
54	6.5	2.8	4.6	1.5		
55	5.7	2.8	4.5	1.3		
56	6.3	3.3	4.7	1.6		
57	4.9	2.4	3.3	1.0		
58	6.6	2.9	4.6	1.3		
59	5.2	2.7	3.9	1.4		
60	5.0	2.0	3.5	1.0		
61	5.9	3.0	4.2	1.5		
62	6.0	2.2	4.0	1.0		
63	6.1	2.9	4.7	1.4		
64	5.6	2.9	3.6	1.3		
65	6.7	3.1	4.4	1.4		
66	5.6	3.0	4.5	1.5		
67	5.8	2.7	4.1	1.0		
68	6.2	2.2	4.5	1.5		
69	5.6	2.5	3.9	1.1		
70	5.9	3.2	4.8	1.8		
71	6.1	2.8	4.0	1.3		
72	6.3	2.5	4.9	1.5		
73	6.1	2.8	4.7	1.2		
74	6.4	2.9	4.3	1.3		
75	6.6	3.0	4.4	1.4		
76	6.8	2.8	4.8	1.4		
77	6.7	3.0	5.0	1.7		
78	6.0	2.9	4.5	1.5		
79	5.7	2.6	3.5	1.0		
80	5.5	2.4	3.8	1.1		
81	5.5	2.4	3.7	1.0		
82	5.8	2.7	3.9	1.2		

↔	83	6.0	2.7	5.1	1.6
	84	5.4	3.0	4.5	1.5
	85	6.0	3.4	4.5	1.6
	86	6.7	3.1	4.7	1.5
	87	6.3	2.3	4.4	1.3
	88	5.6	3.0	4.1	1.3
	89	5.5	2.5	4.0	1.3
	90	5.5	2.6	4.4	1.2
	91	6.1	3.0	4.6	1.4
	92	5.8	2.6	4.0	1.2
	93	5.0	2.3	3.3	1.0
	94	5.6	2.7	4.2	1.3
	95	5.7	3.0	4.2	1.2
	96	5.7	2.9	4.2	1.3
	97	6.2	2.9	4.3	1.3
	98	5.1	2.5	3.0	1.1
	99	5.7	2.8	4.1	1.3

	Species
50	versicolor
51	versicolor
52	versicolor
53	versicolor
54	versicolor
55	versicolor
56	versicolor
57	versicolor
58	versicolor
59	versicolor
60	versicolor
61	versicolor
62	versicolor
63	versicolor
64	versicolor
65	versicolor

65 versicolor
66 versicolor
67 versicolor
68 versicolor
69 versicolor
70 versicolor
71 versicolor
72 versicolor
73 versicolor
74 versicolor
75 versicolor
76 versicolor
77 versicolor
78 versicolor
79 versicolor
80 versicolor
81 versicolor
82 versicolor
83 versicolor
84 versicolor
85 versicolor
86 versicolor
87 versicolor
88 versicolor
89 versicolor
90 versicolor
91 versicolor
92 versicolor
93 versicolor
94 versicolor
95 versicolor
96 versicolor
97 versicolor
98 versicolor
99 versicolor

23BCA0050



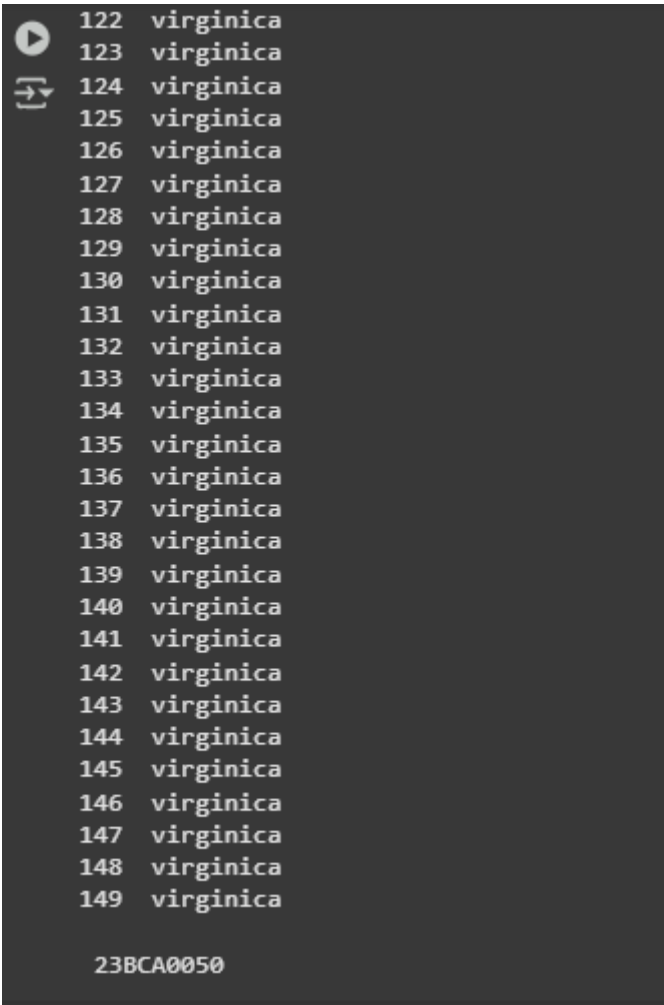
Observations for Iris Virginica:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
100	6.3	3.3	6.0	2.5	
101	5.8	2.7	5.1	1.9	
102	7.1	3.0	5.9	2.1	
103	6.3	2.9	5.6	1.8	
104	6.5	3.0	5.8	2.2	
105	7.6	3.0	6.6	2.1	
106	4.9	2.5	4.5	1.7	
107	7.3	2.9	6.3	1.8	
108	6.7	2.5	5.8	1.8	
109	7.2	3.6	6.1	2.5	
110	6.5	3.2	5.1	2.0	
111	6.4	2.7	5.3	1.9	
112	6.8	3.0	5.5	2.1	
113	5.7	2.5	5.0	2.0	
114	5.8	2.8	5.1	2.4	
115	6.4	3.2	5.3	2.3	
116	6.5	3.0	5.5	1.8	
117	7.7	3.8	6.7	2.2	
118	7.7	2.6	6.9	2.3	
119	6.0	2.2	5.0	1.5	
120	6.9	3.2	5.7	2.3	
121	5.6	2.8	4.9	2.0	
122	7.7	2.8	6.7	2.0	
123	6.3	2.7	4.9	1.8	
124	6.7	3.3	5.7	2.1	
125	7.2	3.2	6.0	1.8	
126	6.2	2.8	4.8	1.8	
127	6.1	3.0	4.9	1.8	
128	6.4	2.8	5.6	2.1	
129	7.2	3.0	5.8	1.6	
130	7.4	2.8	6.1	1.9	
131	7.9	3.8	6.4	2.0	
132	6.4	2.8	5.6	2.2	
133	6.3	2.8	5.1	1.5	
134	6.1	2.6	5.6	1.4	
135	7.7	3.0	6.1	2.3	

↕	135	7.7	3.0	6.1	2.3
	136	6.3	3.4	5.6	2.4
	137	6.4	3.1	5.5	1.8
	138	6.0	3.0	4.8	1.8
	139	6.9	3.1	5.4	2.1
	140	6.7	3.1	5.6	2.4
	141	6.9	3.1	5.1	2.3
	142	5.8	2.7	5.1	1.9
	143	6.8	3.2	5.9	2.3
	144	6.7	3.3	5.7	2.5
	145	6.7	3.0	5.2	2.3
	146	6.3	2.5	5.0	1.9
	147	6.5	3.0	5.2	2.0
	148	6.2	3.4	5.4	2.3
	149	5.9	3.0	5.1	1.8

Species

100 virginica
 101 virginica
 102 virginica
 103 virginica
 104 virginica
 105 virginica
 106 virginica
 107 virginica
 108 virginica
 109 virginica
 110 virginica
 111 virginica
 112 virginica
 113 virginica
 114 virginica
 115 virginica
 116 virginica
 117 virginica
 118 virginica
 119 virginica
 120 virginica
 121 virginica



A screenshot of a Jupyter Notebook interface. On the left, there are icons for running a cell (a play button) and inserting a new cell (a plus sign in a square). The main area displays a list of 28 entries, each consisting of a line number followed by the word 'virginica'. The line numbers range from 122 to 149. At the bottom of the notebook, the identifier '23BCA0050' is visible.

```
122 virginica
123 virginica
124 virginica
125 virginica
126 virginica
127 virginica
128 virginica
129 virginica
130 virginica
131 virginica
132 virginica
133 virginica
134 virginica
135 virginica
136 virginica
137 virginica
138 virginica
139 virginica
140 virginica
141 virginica
142 virginica
143 virginica
144 virginica
145 virginica
146 virginica
147 virginica
148 virginica
149 virginica
```

23BCA0050

4. Write a python program using sciki learn library to predict house prices based on size

PROGRAM :

Step 1: Import necessary libraries

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Prepare the dataset (size of the house vs price)

```
# Example data: New house sizes in square feet and their respective prices
data = {
    'Size': [600, 950, 1200, 1500, 1800, 2100, 2400, 2700, 3000, 3300, 4000],
    'Price': [120000, 145000, 160000, 190000, 220000, 240000, 260000,
280000, 300000, 320000, 400000]
}
```

```
# Convert the dictionary into a pandas DataFrame
df = pd.DataFrame(data)
```

```
# X represents the independent variable (size of the house)
```

```
# Y represents the dependent variable (price of the house)
```

```
X = df[['Size']] # Features (independent variable)
```

```
Y = df['Price'] # Target (dependent variable)
```

```
# Step 3: Split the data into training and test sets
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2,
random_state=0)
```

```
# Step 4: Create and train the linear regression model
```

```
model = LinearRegression()
```

```
model.fit(X_train, Y_train) # Train the model using training data
```

```
# Step 5: Make predictions using the test set
```

```
Y_pred = model.predict(X_test)
```

```
# Step 6: Evaluate the model
```

```
print("Mean squared error:", mean_squared_error(Y_test, Y_pred))  
print("R-squared score:", r2_score(Y_test, Y_pred))
```

Step 7: Visualize the results (optional, but helpful)

```
plt.scatter(X, Y, color='blue') # Plot the actual data points  
plt.plot(X, model.predict(X), color='red') # Plot the regression line  
plt.title('House Size vs Price')  
plt.xlabel('Size of House (in sq. ft.)')  
plt.ylabel('Price of House (in $)')  
plt.show()
```

Step 8: Predict a new price for a given house size

```
house_size = 2500 # Example: size of the house is 2500 sq. ft.  
predicted_price = model.predict([[house_size]]) # Predict the price for this  
size  
print(f"Predicted price for a {house_size} sq. ft. house:  
${predicted_price[0]:,.2f} \n\n 23BCA0050")
```

OUTPUT :

Mean squared error: 88095682.82785647
R-squared score: 0.9797739503711554



Predicted price for a 2500 sq. ft. house: \$269,540.71

23BCA0050